

50 SQL Interview Questions & Answers

Trang 1/10



1. KIẾN THỨC SQL CƠ BẢN

1 SQL là gì?

→ SQL (Structured Query Language) là ngôn ngữ chuẩn dùng để **lưu trữ**, **truy xuất**, **thao tác** và **quản lý** dữ liệu trong các cơ sở dữ liệu quan hệ.

2 Cơ sở dữ liệu (Database) là gì?

→ Một cơ sở dữ liệu là tập hợp có tổ chức các dữ liệu liên quan với nhau, có thể được truy cập, quản lý và cập nhật một cách dễ dàng.



3 Bảng (Table) là gì?

→ Một bảng là tập hợp dữ liệu được tổ chức theo **dòng (rows)** và **cột (columns)**. Đây là cấu trúc cơ bản trong một cơ sở dữ liệu quan hệ.



4 Các loại lệnh SQL là gì?

→ Các lệnh SQL được phân thành 5 loại:

DDL (CREATE, ALTER, DROP, TRUNCATE)	DML (INSERT, UPDATE, DELETE)	DQL (SELECT)	DCL (GRANT, REVOKE)	TCL (COMMIT, ROLLBACK, SAVEPOINT)
---	--	------------------------	-------------------------------	---

5 Sự khác nhau giữa SQL và MySQL là gì?

- **SQL** là ngôn ngữ chuẩn dùng để làm việc với cơ sở dữ liệu quan hệ.
- **MySQL** là một RDBMS cụ thể sử dụng SQL làm ngôn ngữ của nó.



MySQL



Điểm cần nhớ

SQL là công cụ giúp lưu trữ, quản lý và truy xuất dữ liệu hiệu quả từ các cơ sở dữ liệu quan hệ.



Mẹo:

Nắm vững kiến thức cơ bản, các chủ đề nâng cao sẽ dễ dàng hơn rất nhiều! 😊

50 SQL Interview Questions & Answers

Trang 2/10



2. KHÓA & RÀNG BUỘC (KEYS & CONSTRAINTS)

6 Primary Key là gì?

- **Primary Key** là một cột (hoặc tổ hợp nhiều cột) giúp **xác định duy nhất** mỗi dòng trong một bảng.
- Nó không được chứa giá trị **NULL** và phải là **duy nhất**.



PRIMARY KEY

7 Foreign Key là gì?

- **Foreign Key** là một cột tham chiếu đến Primary Key của một bảng khác.
- Nó giúp duy trì **tính toàn vẹn tham chiếu (referential integrity)** giữa các bảng.



FOREIGN KEY

8 Unique Key là gì?

- **Unique Key** là một cột (hoặc tập hợp cột) mà giá trị phải duy nhất cho mỗi dòng.
- Có thể có một giá trị **NULL** (tùy theo DBMS).



UNIQUE KEY

9 Ràng buộc NOT NULL là gì?

- Ràng buộc **NOT NULL** đảm bảo rằng một cột không thể chứa giá trị **NULL**.



10 Sự khác nhau giữa Primary Key và Unique Key là gì?

Đặc điểm	Primary Key	Unique Key
Tính duy nhất	Phải duy nhất	Phải duy nhất
Giá trị NULL	Không được chứa NULL	Có thể chứa một NULL (tùy DBMS)
Số lượng mỗi bảng	Chỉ có một Primary Key	Có thể có nhiều Unique Key
Mục đích	Xác định duy nhất mỗi dòng	Đảm bảo tính duy nhất của giá trị

Ghi chú: Một bảng có thể có nhiều **Unique Key** nhưng chỉ có một **Primary Key**. ★

50 SQL Interview Questions & Answers

Trang 3/10



3. CÁC LỆNH SQL (CRUD)

11) SELECT là gì?

→ Câu lệnh **SELECT** dùng để truy xuất dữ liệu từ một hoặc nhiều bảng.

Ví dụ:

```
SELECT EmpID, Name, Salary
FROM Employees;
```



12) INSERT là gì?

→ Câu lệnh **INSERT** dùng để thêm dòng mới vào một bảng.

Ví dụ:

```
INSERT INTO Employees (EmpID, Name, Salary)
VALUES (101, 'John', 55000);
```



13) UPDATE là gì?

→ Câu lệnh **UPDATE** dùng để sửa đổi các dòng dữ liệu đã có.

Ví dụ:

```
UPDATE Employees
SET Salary = Salary + 5000
WHERE EmpID = 101;
```



14) DELETE là gì?

→ Câu lệnh **DELETE** dùng để xóa các dòng dữ liệu khỏi một bảng.

Ví dụ:

```
DELETE FROM Employees
WHERE EmpID = 101;
```



15) Sự khác nhau giữa DELETE, TRUNCATE và DROP là gì?

Đặc điểm	DELETE	TRUNCATE	DROP
Chức năng	Xóa các dòng khỏi bảng	Xóa toàn bộ dòng khỏi bảng	Xóa hoàn toàn cấu trúc bảng
Mệnh đề WHERE	Có thể dùng WHERE	Không thể dùng WHERE	Không áp dụng
Rollback	Có thể rollback	Thường không thể rollback (tùy DBMS)	Không thể rollback
Ảnh hưởng cấu trúc	Không	Không	Có
Identity / Auto Increment	Không reset	Reset lại Identity	Không áp dụng
Tốc độ	Chậm hơn	Nhanh hơn	Rất nhanh

50 SQL Interview Questions & Answers

Trang 4/10



4. LỌC & SẮP XẾP DỮ LIỆU

16) Mệnh đề WHERE là gì?

→ Mệnh đề **WHERE** dùng để lọc các bản ghi dựa trên một điều kiện cụ thể.

Ví dụ:

```
SELECT *  
FROM Employees  
WHERE Salary > 50000;
```



17) ORDER BY là gì?

→ Mệnh đề **ORDER BY** dùng để sắp xếp tập kết quả theo thứ tự tăng dần hoặc giảm dần.

Ví dụ:

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC;
```



18) DISTINCT là gì?

→ Từ khóa **DISTINCT** dùng để trả về các giá trị duy nhất (distinct).

Ví dụ:

```
SELECT DISTINCT Department  
FROM Employees;
```



Giá trị duy nhất

19) LIMIT / TOP là gì?

→ **LIMIT** (hoặc **TOP** trong một số DBMS) dùng để giới hạn số dòng được trả về.

Ví dụ:

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC  
LIMIT 5;
```

ID	Name
1	----
2	----
3	----
4	----
5	----

Top 5 bản ghi

20) Sự khác nhau giữa WHERE và HAVING là gì?

- **WHERE** lọc các dòng trước khi nhóm (grouping).
- **HAVING** lọc các nhóm sau khi dùng GROUP BY.

WHERE	HAVING
Dùng trước GROUP BY	Dùng sau GROUP BY
Lọc từng dòng riêng lẻ	Lọc theo nhóm
Không thể dùng hàm tổng hợp	Có thể dùng hàm tổng hợp



50 SQL Interview Questions & Answers

Trang 5/10



5. HÀM TỔNG HỢP (AGGREGATE FUNCTIONS)

21) COUNT() là gì?

→ Hàm **COUNT()** dùng để trả về số dòng khớp với một điều kiện.

Ví dụ:

```
SELECT COUNT(*)  
FROM Employees;
```

123

Đếm số dòng

22) SUM() là gì?

→ Hàm **SUM()** dùng để trả về tổng giá trị của một cột số.

Ví dụ:

```
SELECT SUM(Salary)  
FROM Employees;
```



Tổng giá trị

23) AVG() là gì?

→ Hàm **AVG()** dùng để trả về giá trị trung bình của một cột số.

Ví dụ:

```
SELECT AVG(Salary)  
FROM Employees;
```



Giá trị trung bình

24) MIN() và MAX() là gì?

→ **MIN()** trả về giá trị nhỏ nhất trong một cột.

→ **MAX()** trả về giá trị lớn nhất trong một cột.

Ví dụ:

```
SELECT MIN(Salary), MAX(Salary)  
FROM Employees;
```



25) GROUP BY là gì?

→ Mệnh đề **GROUP BY** dùng để nhóm các dòng có cùng giá trị ở các cột được chỉ định.

Ví dụ:

```
SELECT Department, COUNT(*)  
FROM Employees  
GROUP BY Department;
```



Nhóm các bản ghi tương tự

☆ Các hàm tổng hợp (aggregate function) thường được dùng cùng **GROUP BY** để tổng hợp dữ liệu. ❤️

50 SQL Interview Questions & Answers

Trang 6/10



6. SQL JOIN

26 JOIN là gì?

→ **JOIN** dùng để kết hợp các dòng từ hai hoặc nhiều bảng dựa trên một cột liên quan giữa chúng.

Ví dụ:

```
SELECT *  
FROM TableA A  
JOIN TableB B  
ON A.id = B.a_id;
```



Join (Các dòng khớp nhau)

27 INNER JOIN là gì?

→ **INNER JOIN** chỉ trả về các dòng có giá trị khớp nhau ở cả hai bảng.

Ví dụ:

```
SELECT *  
FROM TableA A  
INNER JOIN TableB B  
ON A.id = B.a_id;
```



Inner Join (Chỉ các dòng khớp nhau)

28 LEFT JOIN là gì?

→ **LEFT JOIN** trả về tất cả các dòng từ bảng bên trái và các dòng khớp từ bảng bên phải.

Ví dụ:

```
SELECT *  
FROM TableA A  
LEFT JOIN TableB B  
ON A.id = B.a_id;
```



Left Join (Toàn bộ bảng trái + phần khớp bên phải)

29 RIGHT JOIN là gì?

→ **RIGHT JOIN** trả về tất cả các dòng từ bảng bên phải và các dòng khớp từ bảng bên trái.

Ví dụ:

```
SELECT *  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.id = B.a_id;
```



Right Join (Toàn bộ bảng phải + phần khớp bên trái)

30 FULL OUTER JOIN là gì?

→ **FULL OUTER JOIN** trả về tất cả các dòng khi có sự khớp ở bảng bên trái hoặc bên phải. Nó bao gồm cả các dòng không khớp từ cả hai bảng.

Ví dụ:

```
SELECT *  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.id = B.a_id;
```



Full Outer Join (Toàn bộ các dòng)

☆ Ghi chú: **Cross Join** trả về tích Descartes (Cartesian product) – mỗi dòng của bảng trái kết hợp với mọi dòng của bảng phải. 💖

50 SQL Interview Questions & Answers

Trang 7/10



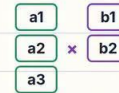
6. SQL JOIN (Tiếp theo)

31) CROSS JOIN là gì?

→ **CROSS JOIN** trả về tích Descartes (Cartesian product) của hai bảng (mỗi dòng của bảng đầu kết hợp với mọi dòng của bảng thứ hai).

Ví dụ:

```
SELECT *  
FROM TableA A  
CROSS JOIN TableB B;
```



Tích Descartes (Mọi tổ hợp)

32) SELF JOIN là gì?

→ **SELF JOIN** là một phép join thông thường trong đó một bảng được join với chính nó.

Ví dụ:

```
SELECT A.name AS Employee,  
       B.name AS Manager  
FROM Employees A  
LEFT JOIN Employees B  
ON A.manager_id = B.id;
```

Cùng một bảng

Employees

33) NATURAL JOIN là gì?

→ **NATURAL JOIN** tự động join các bảng dựa trên các cột có cùng tên và cùng kiểu dữ liệu.

Ví dụ:

```
SELECT *  
FROM TableA  
NATURAL JOIN TableB;
```



Khớp theo tên cột giống nhau

34) NON-EQUI JOIN là gì?

→ **NON-EQUI JOIN** sử dụng các toán tử khác dấu = trong điều kiện join (<, >, <=, >=, <> v.v.).

Ví dụ:

```
SELECT A.name, B.grade  
FROM Employees A  
JOIN SalaryGrade B  
ON A.salary BETWEEN B.min_sal  
AND B.max_sal;
```



Dùng <, >, <=, >=, <> v.v.

35) SEMI JOIN là gì?

→ **SEMI JOIN** trả về các dòng từ bảng bên trái có dòng khớp ở bảng bên phải. Chỉ trả về các cột từ bảng bên trái.

Ví dụ:

```
SELECT A.*  
FROM TableA A  
WHERE EXISTS (SELECT 1  
              FROM TableB B  
              WHERE B.a_id = A.id);
```



Các dòng bảng trái có khớp

50 SQL Interview Questions & Answers

Trang 8/10



7. SUBQUERY & TRUY VẤN LỒNG NHAU

36 Subquery là gì?

→ Một **subquery** là một truy vấn được lồng bên trong một truy vấn khác. Nó có thể được dùng trong mệnh đề SELECT, FROM, WHERE hoặc HAVING.

Ví dụ:

```
SELECT name, salary
FROM Employees
WHERE department_id IN
(SELECT id FROM Departments);
```

Dùng để lọc dữ liệu một cách linh hoạt.

37 Correlated Subquery là gì?

→ Một subquery tham chiếu đến một cột của truy vấn ngoài được gọi là **correlated subquery**.

Ví dụ:

```
SELECT name
FROM Employees e1
WHERE salary > (SELECT AVG(salary)
FROM Employees e2
WHERE e2.department_id = e1.department_id);
```

Thực thi một lần cho mỗi dòng.

38 Toán tử IN với Subquery là gì?

→ Toán tử **IN** dùng để kiểm tra xem một giá trị có tồn tại trong tập kết quả của một subquery hay không.

Ví dụ:

```
SELECT name
FROM Employees
WHERE department_id IN
(SELECT id FROM Departments
WHERE location = 'Bangalore');
```

Departments
id
1
2
3

39 Toán tử EXISTS là gì?

→ **EXISTS** kiểm tra xem subquery có trả về dòng nào không. Trả về TRUE nếu có ít nhất một dòng được trả về.

Ví dụ:

```
SELECT name
FROM Employees e
WHERE EXISTS (SELECT 1
FROM Projects p
WHERE p.emp_id = e.id);
```

40 Scalar Subquery là gì?

→ Một subquery trả về một giá trị duy nhất (một dòng & một cột) được gọi là **scalar subquery**.

Ví dụ:

```
SELECT name,
(SELECT MAX(salary) FROM Employees) AS max_salary
FROM Employees;
```

Trả về một giá trị duy nhất cho truy vấn ngoài.

★ **Mẹo:** Hãy dùng subquery khi cần lọc, so sánh hoặc tính toán giá trị động trong truy vấn. ❤

50 SQL Interview Questions & Answers

Trang 9/10



8. CÁC PHÉP TOÁN TẬP HỢP (SET OPERATIONS)

41 UNION là gì?

→ **UNION** kết hợp tập kết quả của hai truy vấn và loại bỏ các dòng trùng lặp.

Ví dụ:

```
SELECT name FROM TableA
UNION
SELECT name FROM TableB;
```



Toàn bộ các dòng duy nhất

42 UNION ALL là gì?

→ **UNION ALL** kết hợp tập kết quả của hai truy vấn và giữ lại toàn bộ các dòng trùng lặp.

Ví dụ:

```
SELECT name FROM TableA
UNION ALL
SELECT name FROM TableB;
```



Toàn bộ các dòng (kể cả trùng lặp)

43 INTERSECT là gì?

→ **INTERSECT** chỉ trả về các dòng chung giữa hai truy vấn.

Ví dụ:

```
SELECT name FROM TableA
INTERSECT
SELECT name FROM TableB;
```



Các dòng chung

44 EXCEPT / MINUS là gì?

→ **EXCEPT** (hoặc **MINUS** trong một số DBMS) trả về các dòng từ truy vấn đầu tiên mà không có trong truy vấn thứ hai.

Ví dụ:

```
SELECT name FROM TableA
EXCEPT
SELECT name FROM TableB;
```



Các dòng có trong A nhưng không có trong B

45 NOT IN là gì?

→ **NOT IN** trả về các dòng mà giá trị của một cột không tồn tại trong kết quả của một subquery.

Ví dụ:

```
SELECT name FROM Employees
WHERE department_id NOT IN
(SELECT id FROM Departments);
```

46 ANY / SOME là gì?

→ **ANY / SOME** được dùng cùng các toán tử so sánh. Nó trả về TRUE nếu điều kiện đúng với bất kỳ giá trị nào trong subquery.

Ví dụ:

```
SELECT name FROM Employees
WHERE salary > ANY
(SELECT salary FROM Employees
WHERE department_id = 2);
```

So sánh với
từng giá trị và
trả về TRUE
nếu có ít nhất
một giá trị
khớp.

50 SQL Interview Questions & Answers

Trang 10/10



9. CASE STATEMENT & LOGIC ĐIỀU KIỆN

47 CASE Statement là gì?

→ Câu lệnh **CASE** dùng để thực hiện logic điều kiện và trả về các giá trị khác nhau dựa trên điều kiện.

Ví dụ:

```
SELECT name, salary,
CASE
WHEN salary >= 100000 THEN 'High'
WHEN salary BETWEEN 50000 AND 99999 THEN 'Medium'
ELSE 'Low'
END AS Salary_Level
FROM Employees;
```



Rất hữu ích cho việc nhóm & phân loại!

48 COALESCE là gì?

→ **COALESCE** trả về giá trị non-null đầu tiên trong một danh sách biểu thức.

Ví dụ:

```
SELECT name,
COALESCE(phone, mobile, 'Not Available') AS contact_no
FROM Employees;
```



Rất hữu ích khi làm việc với giá trị NULL.

49 NULLIF là gì?

→ **NULLIF** trả về NULL nếu hai biểu thức bằng nhau, ngược lại trả về biểu thức đầu tiên.

Ví dụ:

```
SELECT name,
NULLIF(salary, 0) AS valid_salary
FROM Employees;
```



Giúp tránh lỗi chia cho 0 hoặc so sánh không hợp lệ.

50 IIF / IF() trong SQL là gì?

→ **IIF** (hoặc **IF** trong một số DBMS) là cách viết tắt của CASE, dùng để trả về một trong hai giá trị dựa trên điều kiện.

Ví dụ:

```
SELECT name, salary,
IIF(salary >= 60000, 'Eligible', 'Not Eligible')
AS Bonus_Eligibility
FROM Employees;
```



Nhanh gọn & dễ đọc!

🎯 Những điều cần nhớ

- ★ JOIN kết hợp dữ liệu từ nhiều bảng.
- ★ Các phép toán tập hợp (UNION, INTERSECT, EXCEPT) hoạt động trên tập kết quả.
- ★ Subquery giúp truy vấn trở nên linh hoạt và mạnh mẽ hơn.
- ★ CASE & logic điều kiện giúp đưa ra quyết định thông minh trong truy vấn.
- ★ Hãy luyện tập các khái niệm này với dữ liệu thực tế!

Luyện tập chăm chỉ sẽ thành thạo!

Chúc bạn học SQL vui vẻ!

